

How to Run the Influencer Underwriting Capstone Efficiently and for Free

1. The core principle

You are correct: **a notebook is not a system.**

A notebook is useful for exploration, explanation, and experiments. It becomes dangerous when it is expected to perform all of the following at once:

- download or mount raw data;
- parse millions of records;
- clean data;
- extract images;
- train models;
- save features;
- serve an API;
- monitor drift;
- preserve reproducibility.

The project should instead be organized as a small data and ML platform:

Plain Text

```
Google Drive: immutable raw source + durable artifacts
  ↓
Colab: temporary compute and GPU batch jobs
  ↓
Parquet/DuckDB: compact analytical data products
  ↓
Feature tables and cached embeddings
  ↓
Training and evaluation jobs
  ↓
Small deployment service
  ↓
Monitoring reports and model cards
```

The guiding rule is:

Never repeatedly scan or unzip the full raw dataset when a compact derived table can answer the question.

2. Free execution strategy

The most practical no-cost setup is:

Component	Recommended free role
Google Drive	Source of truth for archives, checkpoints, Parquet files, reports, and model artifacts
Google Colab CPU	Audits, metadata extraction, DuckDB/Parquet transformations, classical ML
Google Colab GPU	Text/image embedding extraction and selected deep-learning experiments
GitHub	Source code, configuration, documentation, and version control
Local laptop	Editing, Git, small tests, API development, and dashboard development—not full raw-data processing
DuckDB	Querying Parquet without loading everything into RAM
PyArrow/Parquet	Compact columnar storage and partitioned feature tables
MLflow local files or experiment logs	Model and run tracking without requiring a server
FastAPI	Small scoring API
Streamlit	Demonstration dashboard
Docker	Reproducible deployment if available locally; otherwise use a documented Python environment

Free notebook quotas, RAM, disk, and GPU availability vary. Treat Colab as **ephemeral compute**, not as permanent storage. Any important output must be copied to Drive or GitHub before the session ends.

3. What should remain on Google Drive

Keep the original archives untouched.

Recommended Drive structure:

Plain Text

```
InfluencerUnderwriting/
├── raw/
│   ├── post_info.txt
│   ├── json_files.zip
│   ├── profiles_influencers.zip
│   ├── profiles_brands.zip
│   ├── img_fi01.zip ... img_fi16.zip
│   └── sample_images.zip
├── metadata/
│   ├── data_dictionary.yaml
│   ├── schema_report.json
│   ├── coverage_report.parquet
│   └── join_validation_report.json
├── bronze/
│   ├── posts_metadata.parquet/
│   ├── profiles_influencers.parquet/
│   ├── profiles_brands.parquet/
│   └── post_image_index.parquet/
├── silver/
│   ├── posts_features.parquet/
│   ├── creator_features.parquet/
│   ├── brand_features.parquet/
│   └── creator_brand_edges.parquet/
├── embeddings/
│   ├── text/
│   ├── image/
│   └── multimodal/
├── models/
│   ├── sponsorship/
│   ├── engagement/
│   ├── matching/
│   └── anomaly/
├── evaluations/
│   ├── metrics/
│   ├── plots/
│   ├── predictions/
│   └── error_analysis/
└── reports/
    ├── data_card.md
    └── model_cards/
```

```
|— monitoring/  
|— final_capstone_report.md
```

The directories can be created once using a small setup script. Do not use Drive as a temporary extraction directory for millions of small JSON files. Drive-mounted file operations are slow and can create a large number of metadata operations.

4. The most important storage decision: convert raw data to Parquet

The raw data is archive-oriented: text index files, zip files, JSON files, and image archives. The modeling system should be table-oriented.

Bronze tables

Create these once:

posts_metadata.parquet

One row per post, containing fields such as:

- `post_id` ;
- `username` ;
- `sponsored` ;
- `json_file` ;
- `n_images` ;
- image filenames or a separate image index;
- timestamp if extracted;
- likes and comments if available;
- caption if available;
- location and tag counts;
- JSON schema/layout flag.

profiles_influencers.parquet

One row per influencer:

- `username`;
- `followers`;

- followees;
- profile post count;
- category;
- bio;
- profile URL;
- profile image reference;
- missingness flags.

Do not include email or phone fields in ordinary modeling tables unless there is a documented, authorized reason. They are not required for the recommender and create unnecessary privacy risk.

profiles_brands.parquet

One row per brand:

- brand username;
- category;
- bio;
- followers;
- posts;
- profile availability flag.

post_image_index.parquet

One row per post-image relationship:

- post ID;
- JSON filename;
- image filename;
- image order;
- archive/shard name;
- image existence flag.

Silver tables

These are compact derived tables used by almost all models:

- `posts_features.parquet` ;

- `creator_features.parquet` ;
- `brand_features.parquet` ;
- `creator_brand_edges.parquet` ;
- `campaign_training_examples.parquet` .

Once these exist, most analysis should query the silver tables instead of touching the raw archives.

5. Do not load 1.6 million JSON objects into a Python list

The first implementation should be a streaming extractor.

Bad pattern

Python

```
all_records = []
for name in zip_file.namelist():
    all_records.append(json.load(...))
```

This creates avoidable memory pressure and often crashes a free Colab runtime.

Better pattern

1. Read `post_info.txt` in chunks.
2. Build a small set of required JSON filenames.
3. Stream the ZIP archive.
4. Parse one JSON object at a time.
5. Write batches of perhaps 5,000–25,000 rows to Parquet.
6. Delete Python objects after each batch.
7. Save a checkpoint after each shard or batch.

The first extraction should not attempt to preserve every nested JSON field. Extract only the fields required for the current project.

Suggested first-pass fields

Plain Text

```
post_id
username
sponsored
json_file
caption
created_at / timestamp
likes
comments
comments_disabled
location
n_tagged_users
n_comments
has_disclosure_hashtag
json_layout
n_images
image_available
```

If a field is absent in a JSON layout, write null and record the layout flag. Do not fail the entire batch because one optional field is missing.

6. Use DuckDB for analysis without loading the dataset into RAM

DuckDB can query Parquet directly and aggregate large tables efficiently.

Typical workflow:

SQL

```
SELECT
    username,
    COUNT(*) AS n_posts,
    AVG(LOG1P(likes)) AS mean_log_likes,
    AVG(LOG1P(comments)) AS mean_log_comments,
    AVG(sponsored) AS sponsored_rate
FROM read_parquet('silver/posts_features.parquet')
GROUP BY username;
```

Write the result to a smaller table:

SQL

```
COPY (
    SELECT
        username,
```

```
COUNT(*) AS n_posts,  
AVG(LOG1P(likes)) AS mean_log_likes,  
AVG(LOG1P(comments)) AS mean_log_comments,  
AVG(sponsored) AS sponsored_rate  
FROM read_parquet('silver/posts_features.parquet')  
GROUP BY username  
) TO 'silver/creator_features.parquet' (FORMAT PARQUET, COMPRESSION ZSTD);
```

Use Parquet compression such as Zstandard. It reduces storage and makes repeated analytical scans faster.

7. The staged execution plan

Stage 0: Repository and environment setup

Create a Git repository with this layout:

Plain Text

```
influencer-underwriting/  
├─ README.md  
├─ pyproject.toml  
├─ requirements.txt  
├─ .gitignore  
├─ configs/  
│   ├── paths.yaml  
│   ├── features.yaml  
│   ├── campaigns.yaml  
│   └─ models.yaml  
├─ src/  
│   └─ underwriting/  
│       ├── io/  
│       ├── schemas/  
│       ├── extraction/  
│       ├── features/  
│       ├── models/  
│       ├── ranking/  
│       ├── monitoring/  
│       └─ api/  
└─ scripts/  
    ├── audit_raw.py  
    ├── extract_metadata.py  
    ├── build_features.py  
    ├── train_models.py  
    └─ evaluate.py
```

```
|   └─ monitor.py
└─ notebooks/
    │   └─ 01_audit_report.ipynb
    │   └─ 02_exploration.ipynb
    │   └─ 03_model_analysis.ipynb
    │   └─ 04_demo.ipynb
└─ tests/
└─ dashboard/
└─ Dockerfile
└─ docker-compose.yml
```

The rule is:

Production logic lives in `src/` and `scripts/`. Notebooks call that logic and visualize results.

A notebook should be restartable from a clean runtime and should not contain hidden state or manually edited outputs.

Stage 1: Raw-data audit

Run this only when the raw data changes.

The audit should produce:

- file inventory;
- archive member counts;
- JSON schema/layout frequencies;
- post count and ID integrity;
- sponsorship balance;
- profile coverage;
- image coverage;
- missing field rates;
- duplicate checks;
- timestamp range;
- category distribution;
- rich-layout comment coverage;
- post-to-JSON and post-to-image join rates.

The three checks from your notes should be formal gates before modeling:

1. What proportion of JSON files uses the richer layout, and how many posts have comment text?
2. How many creators have at least five sponsored and five organic posts?
3. How many creators and posts map to each client vertical?

The output should be `metadata/audit_manifest.json` and a human-readable HTML or Markdown report.

Why the “five sponsored and five organic posts” check matters

The audited dataset has a median of approximately 24 posts per creator and an overall sponsorship rate of 13.85%. A typical creator may have only around three sponsored posts, and sponsorship may be concentrated among a smaller group.

Therefore, sponsored-to-organic retention ratios will be noisy for many creators. Use minimum evidence thresholds and shrinkage rather than ranking every creator equally.

Stage 2: Metadata extraction without images

Do this before using GPU resources.

Extract all useful metadata into Parquet. This stage may be CPU-heavy but does not need a GPU.

Recommended output:

Plain Text

```
bronze/posts_metadata.parquet
bronze/profiles_influencers.parquet
bronze/profiles_brands.parquet
bronze/post_image_index.parquet
```

At this point you should be able to answer most data-understanding questions without unpacking the 189 GB image collection.

Use a small sample for schema discovery

Before processing all JSON records:

1. Parse 500–2,000 JSON files.
2. Count field paths and layouts.
3. Inspect examples for each layout.

4. Define explicit extraction functions.
5. Test on a fixed sample.
6. Only then launch the full streaming job.

This avoids discovering a schema problem after several hours of processing.

Stage 3: Initial exploration and cleaning

Use DuckDB, pandas, Polars, or PyArrow against Parquet.

Produce:

- follower distribution plots on a log scale;
- engagement distribution plots on a log scale;
- sponsored versus organic counts;
- creator posting-volume distribution;
- category counts;
- category and vertical coverage;
- profile-to-post consistency checks;
- image coverage by category;
- timestamp and activity plots;
- missingness report.

Cleaning rules

- Convert counts to numeric with invalid-value tracking.
 - Use `log1p` for followers, likes, comments, and post counts.
 - Preserve raw values and create cleaned columns.
 - Treat `NULL` category as missing, not as a meaningful niche.
 - Normalize category names and spelling.
 - Keep an explicit `low_evidence` flag.
 - Do not silently drop creators with missing fields.
 - Keep image availability as a feature and a reporting dimension.
 - Store every cleaning decision in `data_dictionary.yaml`.
-

Stage 4: Build the classical feature store

Start with a strong, lightweight feature store before deep learning.

Post-level feature groups

Profile and scale

- log followers;
- log followees;
- category;
- profile post count;
- creator posting volume.

Engagement

- log likes;
- log comments;
- comments-to-likes ratio;
- likes-per-follower proxy;
- comments-per-follower proxy;
- comment-disabled indicator.

Commercial

- sponsorship label;
- disclosure hashtag;
- sponsor mention;
- product/brand mention count;
- caption commercial-language features.

Time

- year/month/week;
- day of week;
- hour if timestamp and timezone are usable;
- creator-relative posting interval;

- campaign-period features.

Text

- caption length;
- hashtag count;
- mention count;
- emoji count;
- language;
- sentiment or emotion;
- topic IDs;
- TF-IDF or sentence embeddings.

Images

Initially use:

- number of images;
- image availability;
- image dimensions if available;
- image embedding only for a staged sample.

Creator-level features

Aggregate by creator using robust statistics:

- median rather than only mean engagement;
- quantiles;
- interquartile range;
- top-decile rate;
- sponsored rate;
- organic engagement;
- sponsored engagement;
- posting consistency;
- number of brands mentioned;
- content-topic distribution;
- evidence count;

- anomaly score.

Do not calculate a creator's score from one viral post.

Stage 5: Build the potential buyer-response index

The concept from your notes is useful, but it must be labeled honestly.

A defensible index is:

Plain Text

```
PotentialBuyerResponseIndex
= follower_ceiling_proxy
× expected_engaged_share
× purchase_intent_proxy
× sponsored_retention
× timing_effect
- compliance_and_safety_penalty
```

Because real viewers and buyers are not observed, each term must be labeled as a proxy.

Suggested terms

Follower ceiling proxy

Use a log-transformed and capped follower measure. Do not allow the largest accounts to dominate solely because of scale.

Expected engaged share

Use creator-normalized likes and comments, preferably estimated out-of-sample.

Purchase-intent proxy

Use available caption/comment text only if the dataset has usable comment text. Search for signals such as:

- price questions;
- availability questions;
- “where can I buy” language;
- link or code requests;
- product specifications;

- recommendation requests.

If comment text is unavailable for most posts, rename this term to **commercial-interest language proxy** rather than purchase intent.

Sponsored retention

Compare sponsored engagement with a creator's organic baseline using shrinkage and minimum evidence thresholds.

Timing effect

Use creator-specific historical patterns, but do not claim causality.

Penalty

Apply penalties for:

- brand-safety flags;
- high anomaly score;
- excessive sponsorship;
- low evidence;
- category mismatch;
- missing critical fields.

Avoid false precision

Report the index with:

- evidence count;
- confidence band;
- low-data warning;
- component scores;
- rank stability.

A score of 78.4 should not imply that the system has measured 78.4% buyer probability.

Stage 6: Baseline models first

Before GPU work, implement these models:

Model 1: Disclosure classifier

Target: explicit disclosure or sponsorship label.

Baselines:

- disclosure keyword rules;
- logistic regression with TF-IDF;
- gradient boosting with metadata;
- transformer text model later.

Model 2: Engagement model

Target:

- log likes;
- log comments;
- top-decile engagement.

Baselines:

- follower-count-only model;
- creator-median baseline;
- linear regression;
- gradient boosting;
- quantile regression.

The follower-only model is essential because it provides a simple benchmark for whether the sophisticated system adds value.

Model 3: Creator category or niche model

Target:

- profile category;
- soft niche memberships;
- campaign vertical fit.

Model 4: Anomaly model

Use:

- robust z-scores;
- isolation forest;
- creator-relative engagement residuals;

- posting burstiness.

Model 5: Creator ranking model

Use a weighted score first, then compare against:

- follower-count ranking;
 - engagement-rate ranking;
 - random eligible ranking;
 - human ranking if available.
-

Stage 7: GPU work only after the feature store is stable

GPU sessions should be used for reusable outputs, not one-off notebook cells.

Good GPU jobs

- Generate caption embeddings once.
- Generate image embeddings once.
- Fine-tune a sponsorship classifier.
- Train a multimodal engagement or campaign-fit model.
- Build vector indexes from cached embeddings.

Bad GPU jobs

- Re-reading Drive and decoding the same images repeatedly.
- Extracting all 189 GB of images before validating the pipeline.
- Running a large model without a saved checkpoint.
- Training on a random split and reporting only accuracy.
- Keeping all image tensors in RAM.

Image strategy

Use three phases:

Phase A: proof of concept

Use a stratified image sample across categories, sponsorship labels, follower tiers, and image counts.

Phase B: cached embeddings

Extract embeddings in batches, save them to sharded files, and record:

- post ID;
- image filename;
- model name and version;
- embedding dimension;
- extraction date;
- failed-image status.

Phase C: selective expansion

Only expand to the full image archive if the sample results demonstrate clear value over metadata and text.

This is both computationally efficient and academically stronger because it gives an ablation analysis.

Stage 8: Correct evaluation design

The evaluation must simulate future use.

Required splits

Random post split

Use as a baseline only.

Creator-held-out split

Hold out complete creators. Tests whether the model generalizes to new influencers.

Time-held-out split

Train on earlier posts, test on later posts. Tests deployment-like behavior.

Creator-and-time-held-out split

Best for an agency selecting creators for future campaigns.

Sponsorship leakage experiment

Report four variants:

1. Full text.
2. Disclosure terms included.
3. Disclosure terms masked.
4. Text plus image plus metadata with disclosure terms masked.

This demonstrates that you understand leakage, label construction, and model interpretation.

Metrics

Classification

- precision;
- recall;
- F1;
- PR-AUC;
- ROC-AUC;
- calibration;
- confusion matrix.

Regression

- MAE on log engagement;
- RMSE;
- Spearman correlation;
- prediction interval coverage.

Ranking

- Precision@K;
- Recall@K;
- NDCG@K;
- pairwise agreement;
- ranking stability;
- uplift over follower-count baseline.

Fairness and coverage

Report by:

- follower tier;
 - creator category;
 - sponsored versus organic;
 - high- and low-evidence creators;
 - image-covered versus image-missing posts.
-

Stage 9: Build the niche map

A niche should not be only a profile category. Define a working niche as:

Plain Text

```
niche = content topic × client vertical
```

For example:

- food × restaurant;
- food × kitchen appliances;
- interior × kitchen appliances;
- family × household products;
- productivity × software;
- automobile content × used cars.

Each niche cell should report:

- number of creators;
- number of posts;
- number of sponsored posts;
- median organic engagement;
- median sponsored engagement;
- credibility-weighted retention;
- commercial-interest evidence;
- saturation or brand concentration;
- confidence level.

A “working niche” should require:

- enough creators;
- enough posts;
- adequate sponsored and organic evidence;
- strong content fit;
- acceptable risk;
- stable estimates.

If software and used-car niches are thin, present them as **extensibility demonstrations**, not as statistically established conclusions.

Stage 10: Add constrained assignment

The agency does not merely need a ranking. It needs to assign creators to clients and time windows.

Inputs

- campaign vertical;
- number of influencers required;
- time window;
- target niche;
- minimum score;
- maximum sponsorship fatigue;
- safety threshold;
- optional budget and exclusivity constraints.

Output

- ranked shortlist;
- assigned creators;
- backup creators;
- reasons for inclusion;
- reasons for exclusion;
- expected score range;
- evidence count.

Start with a greedy constrained ranking. Later, if budget and capacity constraints are supplied, compare with:

- integer programming;
- linear assignment;
- portfolio optimization;
- diversity-aware re-ranking.

Do not over-engineer optimization before the scoring features are trustworthy.

Stage 11: Deployment architecture

Only the compact derived tables and trained models should be used at inference time.

Plain Text

```
FastAPI /score
  ↓
Campaign brief validation
  ↓
Feature store + vector index
  ↓
Candidate retrieval
  ↓
Model predictions
  ↓
Underwriting score
  ↓
Explanation evidence
```

Suggested endpoints

Plain Text

```
POST /score
POST /shortlist
POST /assign
GET /niches
GET /memo/{creator_id}
GET /health
GET /drift
```

The API should not open the 3 GB JSON archive or 189 GB image archive during a request. It should read compact feature tables and cached embeddings.

Dashboard pages

1. Campaign brief intake.
2. Niche map.
3. Candidate shortlist.
4. Creator underwriting memo.
5. Assignment calendar.
6. Model evidence and confidence.
7. Monitoring and drift.

For a free class demonstration, run FastAPI and Streamlit locally or in a short-lived Colab session. Do not promise a permanently available production service from a free, ephemeral notebook runtime.

Stage 12: Monitoring without expensive infrastructure

You can demonstrate serious MLOps with scheduled or manually triggered reports rather than a large monitoring cluster.

Data monitoring

Track:

- row counts;
- null rates;
- category distribution;
- follower distribution;
- sponsorship rate;
- image coverage;
- timestamp range;
- embedding failures;
- duplicate rate.

Model monitoring

Track:

- performance on a fixed holdout set;
- calibration;
- score distribution;
- category-level performance;
- recommendation coverage;
- percentage of low-confidence outputs;
- drift in input embeddings;
- drift in engagement outcomes.

Simple monitoring implementation

Every run writes:

Plain Text

```
reports/monitoring/YYYY-MM-DD.json  
reports/monitoring/YYYY-MM-DD.html
```

Compare the current report with a baseline. Trigger a warning when a threshold is exceeded.

Examples:

- sponsorship rate changes by more than a selected tolerance;
- category distribution changes sharply;
- image coverage drops;
- model confidence becomes unusually high or low;
- a category has too few eligible creators;
- ranking becomes dominated by follower count.

8. How to use Colab efficiently

One Colab notebook per job type

Do not keep one enormous notebook. Use:

1. `01_audit_raw.ipynb` — runs once or when raw files change.

2. `02_extract_metadata.ipynb` — calls the streaming extractor.
3. `03_build_features.ipynb` — creates Parquet features.
4. `04_train_classical.ipynb` — fast baselines.
5. `05_extract_embeddings_gpu.ipynb` — GPU batch job.
6. `06_train_multimodal.ipynb` — GPU model training.
7. `07_evaluate_and_report.ipynb` — results only.
8. `08_demo_app.ipynb` — optional interactive demonstration.

Each notebook should call scripts from the repository.

Session startup sequence

At the top of each Colab notebook:

1. Mount Drive.
2. Clone or pull the Git repository.
3. Install pinned dependencies.
4. Define `PROJECT_ROOT`, `RAW_ROOT`, and `ARTIFACT_ROOT`.
5. Check whether the required artifact already exists.
6. Skip completed stages.
7. Run a small smoke test.
8. Launch the requested job.

Drive and local runtime separation

Use Drive for durable files. Use `/content` for temporary high-speed work.

Plain Text

```
Drive: raw archives, Parquet outputs, checkpoints
/content: current batch, extracted temporary images, caches, logs
```

Do not write millions of tiny output files directly to Drive. Write a local batch, combine it, and copy a compact artifact to Drive.

Checkpoint every meaningful unit

Checkpoint after:

- each ZIP shard;

- every fixed number of JSON records;
- every image embedding batch;
- every training epoch or evaluation fold.

A resumed job should detect completed shards and continue instead of starting over.

9. What not to do

Avoid these common failure modes:

Do not unzip all images into Google Drive

The image archive is extremely large and contains many files. This creates slow I/O and storage pressure.

Do not process all images before metadata analysis

Metadata, captions, profiles, sponsorship labels, and engagement can answer many questions without image decoding.

Do not train directly from nested JSON

Convert to flat, typed, versioned Parquet tables first.

Do not use follower count as the complete recommendation

Use it as a baseline and one component of the underwriting score.

Do not use random splits as the only evaluation

Creator leakage will make results look better than future performance.

Do not use comments or likes as pre-publication features

They are valid targets or post-publication monitoring signals, but not valid inputs for a pre-publication recommendation unless the use case explicitly scores an already-published post.

Do not call the output “probability of purchase”

Call it potential buyer-response exposure, qualified engagement potential, or campaign suitability unless real conversion labels are later added.

Do not deploy raw data access in the API

The deployment layer should use compact artifacts only.

10. The first three implementation gates

Before committing to advanced modeling, run the following three analyses.

Gate 1: Rich-layout and comment coverage

Measure:

- percentage of JSON files using the richer layout;
- percentage containing comment text;
- number of comments or comment-like records;
- language coverage;
- missingness by layout.

Decision:

- If comment text is broadly available, build a purchase-intent proxy.
- If comment text is sparse, use caption and visible-engagement signals only and rename the target accordingly.

Gate 2: Sponsored/organic evidence per creator

Count creators with:

- at least five sponsored posts and five organic posts;
- at least three sponsored and three organic posts;
- any sponsored posts;
- enough total posts for stable engagement estimation.

Decision:

- Use shrinkage and confidence weighting.
- Report how much of the creator population is eligible for retention analysis.
- Do not rank creators with insufficient evidence as if they were equally reliable.

Gate 3: Client-vertical coverage

Count creators and posts for:

- food and restaurants;

- home, interior, and appliances;
- software, apps, and technology;
- automobiles and dealers;
- other selected verticals.

Decision:

- Build strongest claims around well-supported verticals.
- Present thin verticals as transfer or extensibility demonstrations.
- Use text and image semantic matching to expand beyond profile categories.

11. Recommended order of work

The efficient order is:

Plain Text

1. Audit raw data
2. Extract metadata only
3. Convert to Parquet
4. Build DuckDB queries
5. Create data dictionary and quality reports
6. Build classical feature store
7. Establish follower-count and creator-median baselines
8. Train classical models
9. Evaluate with creator/time-held-out splits
10. Extract text embeddings
11. Run image sample experiment
12. Extract cached image embeddings if valuable
13. Train multimodal model
14. Build niche map
15. Build underwriting score
16. Build constrained shortlist
17. Add RAG explanation layer
18. Serve compact artifacts through API
19. Add dashboard and monitoring
20. Package report, model cards, and demo

This order prevents expensive modeling before you know that:

- the data joins correctly;
- labels are meaningful;
- the target has enough evidence;

- the vertical has enough coverage;
 - the sophisticated model beats simple baselines.
-

12. Minimum viable capstone versus advanced version

Minimum viable version

A credible first release can include:

- metadata extraction;
- Parquet feature tables;
- creator-level engagement normalization;
- sponsorship/disclosure model;
- niche classification;
- transparent underwriting score;
- shortlist API;
- Streamlit dashboard;
- evaluation against follower-count baseline;
- data card and model card.

Advanced version

Add:

- cached text embeddings;
- cached image embeddings;
- multimodal engagement model;
- graph-based creator–brand matching;
- anomaly detection;
- comment-based commercial-interest model;
- LLM campaign-brief parser;
- RAG explanations;
- constrained assignment optimization;
- monitoring dashboard;
- drift alerts;

- human review workflow.

Do not sacrifice correctness and reproducibility to include every advanced feature. A complete baseline system with honest evaluation is stronger than an unfinished collection of sophisticated models.

13. Final operating model

The complete system should operate like this:

Offline data plane

- Read raw archives from Drive.
- Stream and normalize records.
- Write compressed Parquet.
- Generate creator and post features.
- Cache embeddings.
- Train and evaluate models.
- Save versioned artifacts.

Online decision plane

- Accept a campaign brief.
- Retrieve candidate creators from compact tables and vector indexes.
- Score engagement, fit, commercial readiness, safety, and confidence.
- Apply constraints.
- Produce a shortlist and underwriting memo.

Monitoring plane

- Compare new data and predictions to historical baselines.
- Detect drift and coverage problems.
- Track model performance when new labels become available.
- Record low-confidence decisions for human review.

This is the difference between a notebook and a system:

The notebook shows the work. The pipeline owns the work. The artifacts make the work reusable. The API exposes the work. Monitoring tells you when the work becomes

unreliable.

14. Final recommendation

Use Google Drive as the permanent source and artifact repository, but never treat it as the active compute disk. Use Colab as disposable CPU/GPU workers. Convert the raw archives into compact Parquet tables once, query them with DuckDB, cache text and image embeddings, and make the API depend only on those compact artifacts.

Begin with metadata and classical baselines. Use GPU only after proving that embeddings or multimodal models add value. Evaluate using creator-held-out and time-held-out splits. Keep the potential buyer-response index explicitly proxy-based. Deploy a small scoring service and dashboard rather than attempting to serve the raw corpus.

That architecture is free or near-zero-cost with variable notebook limits, avoids overwhelming your laptop, and demonstrates the full progression expected in an applied machine-learning capstone:

Plain Text

```
data understanding
→ exploration
→ cleaning
→ feature engineering
→ supervised learning
→ unsupervised learning
→ deep learning
→ GenAI/RAG
→ deployment
→ monitoring
```